ETH zürich

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

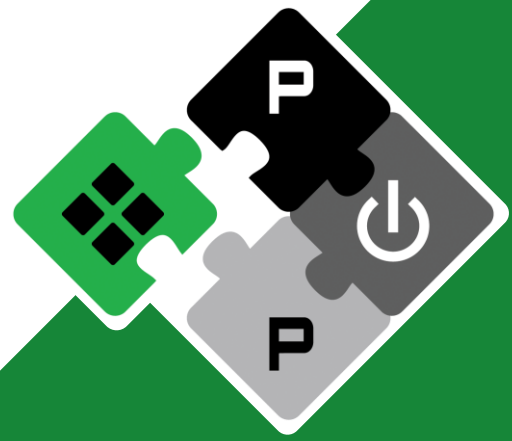
CachePool: Many-core cluster of customizable, lightweight scalar-vector PEs for irregular L2 data-plane workloads

Integrated Systems Laboratory (ETH Zürich)

Zexin Fu, Diyou Shen zexifu, dishen@iis.ee.ethz.ch

Alessandro Vanelli-Coralli avanelli@iis.ee.ethz.ch

Luca Benini lbenini@iis.ee.ethz.ch



PULP Platform

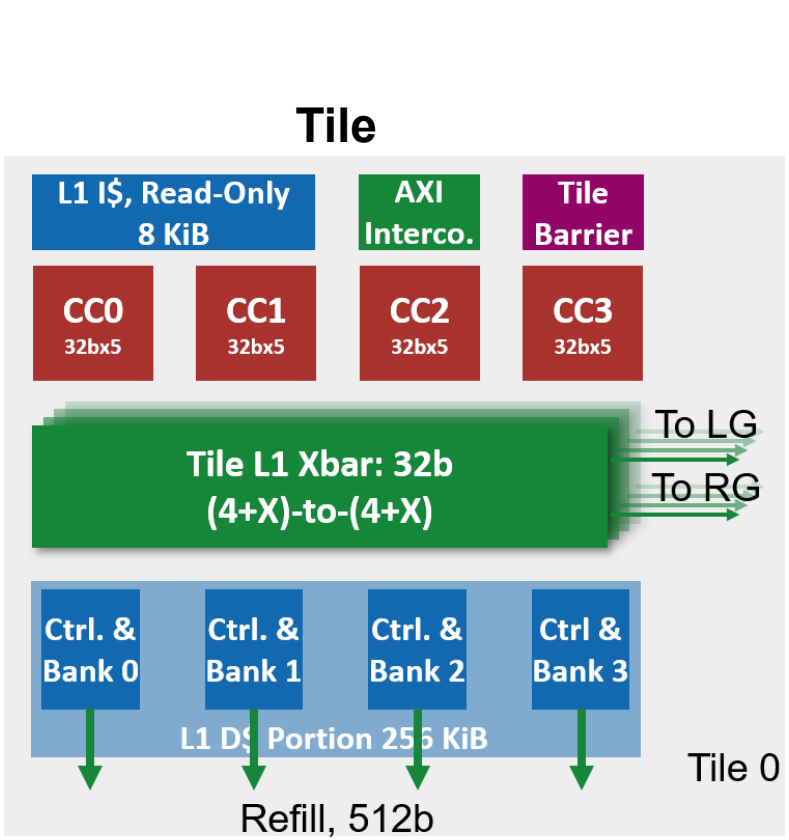
Open Source Hardware, the way it should be!

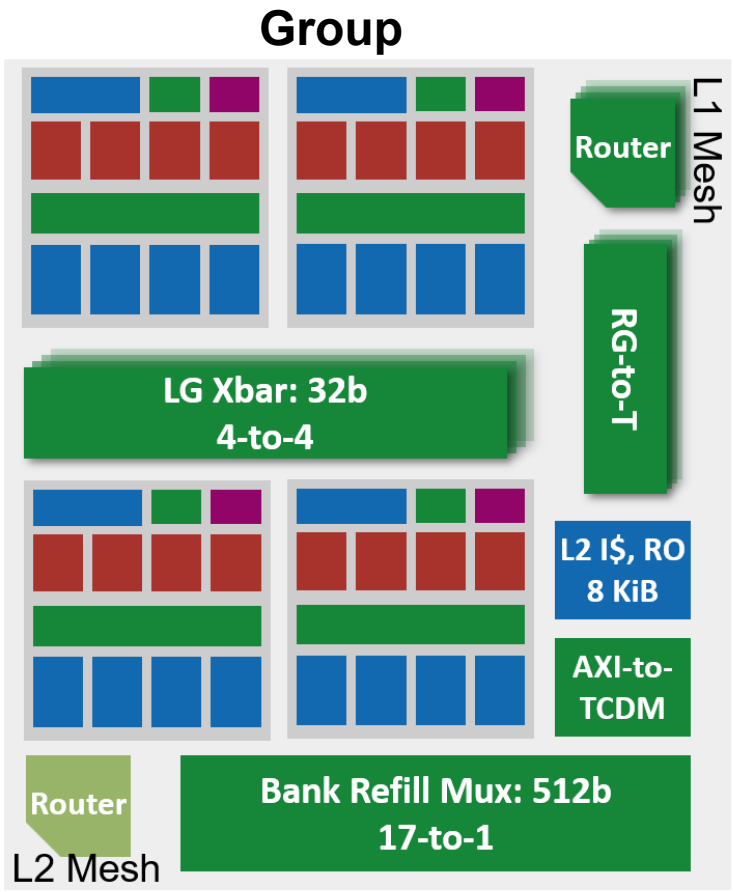
@pulp_platform

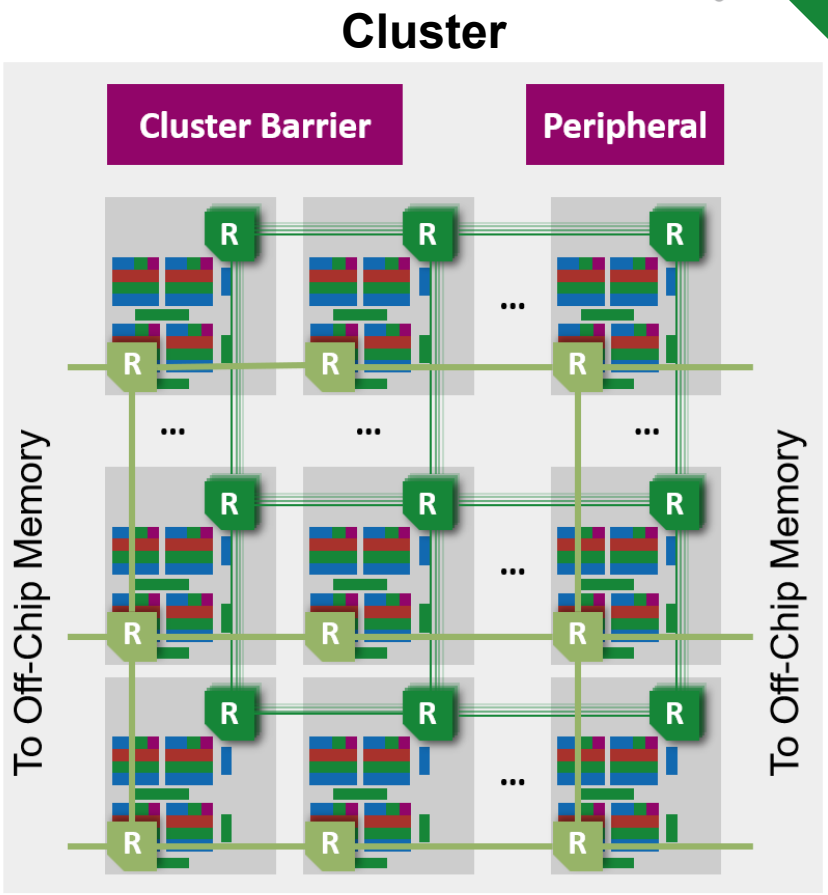
pulp-platform.org

youtube.com/pulp_platform

Hardware Development (Cluster)







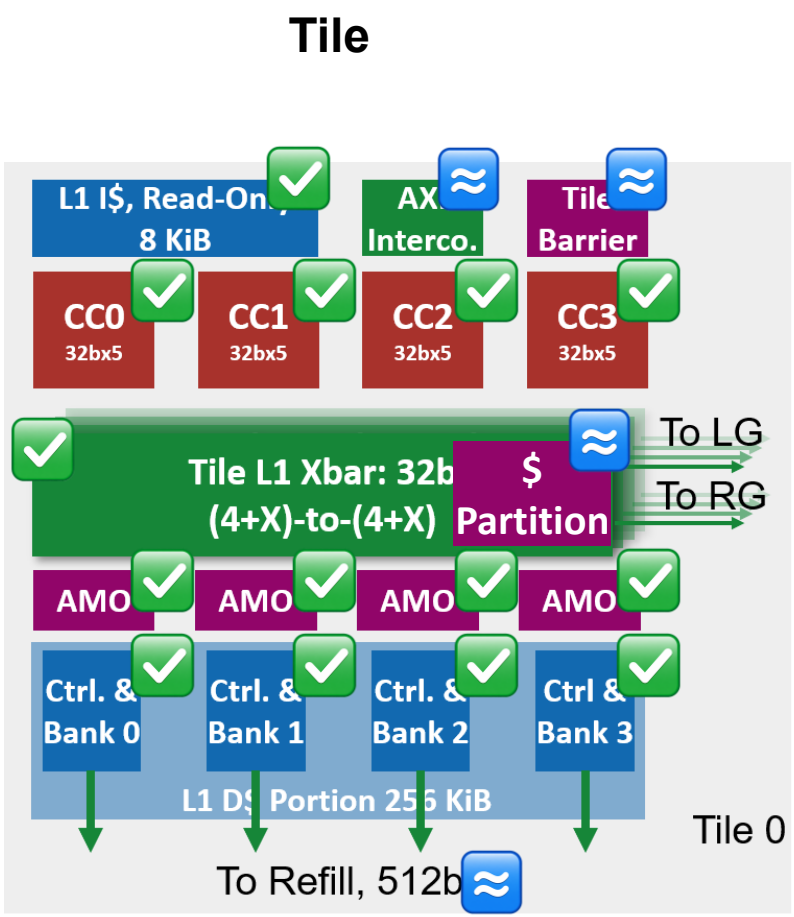
Hardware Development (Cluster)

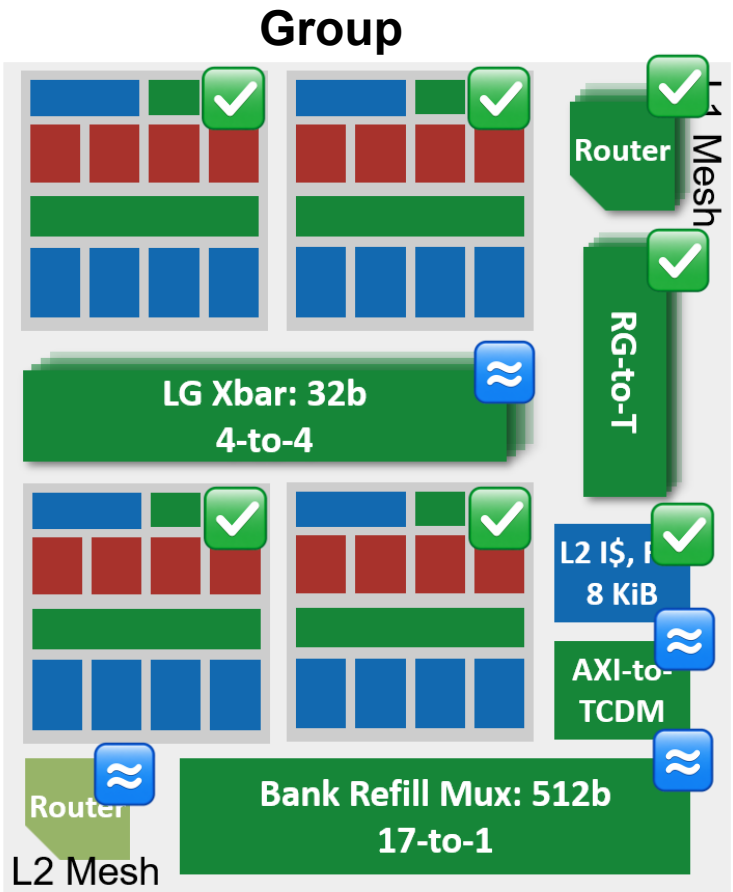
- **Developments**
  - Fix a bug that responses are always go through one router links
    - This bug causes significant traffic congestions
    - Current performance still lower than expected => more investigation undergoing
  - WIP: add multi-scalar core support
  - WIP: rebase to incorporate folded cache

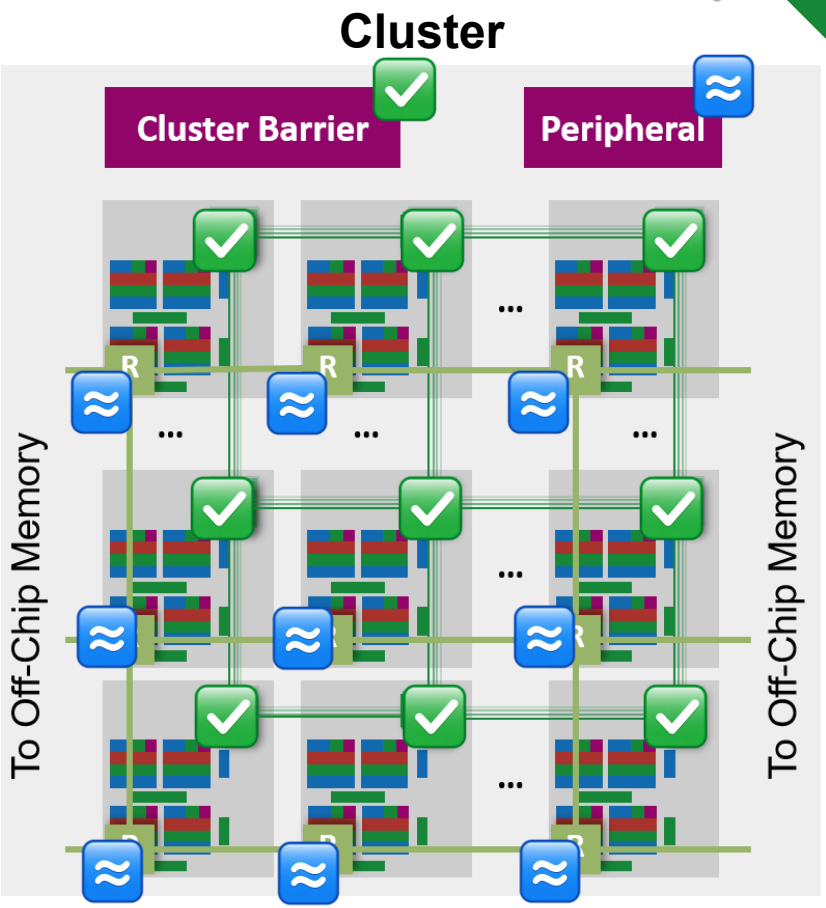
Benchmark Plan

- **Software kernels**
  - Normal testing
    - Sequential/Random RW: for mesh bandwidth/latency analysis
    - dotp: smoke test for vector PEs
  - Control Algorithm
    - GEMV: included in the algorithm
    - GEMM: included in the algorithm
    - Exponential: for log(1+S) part. Taylor expansion or using some non-liner HW extension
    - Combined chain: connect micro-kernels to form an execution chain
  - RLC Packet Handling
    - User case 1,2,3
- **Hardware Configurations**
  - **Single-level cache only**
    - **256S + 256V**
      Baseline configuration, RTL main work finished, under testing and verification
    - **512S + 256V**
      Increased scalar core counts, WIP
  - **Two-level cache**
    - **256S + 256V**
      PR opened, close to ready
    - **512S + 256V**
      Increased scalar core counts, WIP

GVSoC Modeling Progress







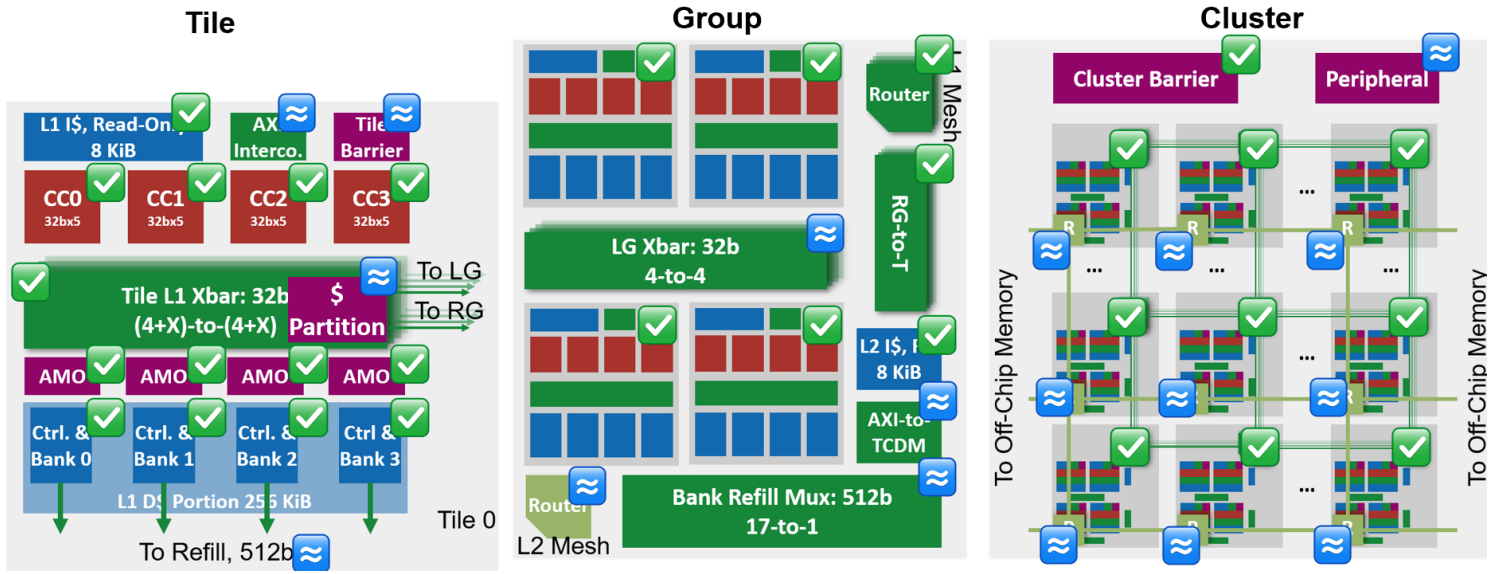
Done, the same logic as RTL

In place, with approximate model

Not implemented yet

GVSoC Modeling Progress

- **Tile level**
  - Add cache partition support
- **Group level**
  - Add the bank refill mux
  - Add the L2 refill NoC router
- **Cluster level**
  - Add the L2 NoC
  - Add the memory channels at the edge of the L2 NoC
- **System level**
  - Interconnect model: Sync → Async (model the contention more realistic)



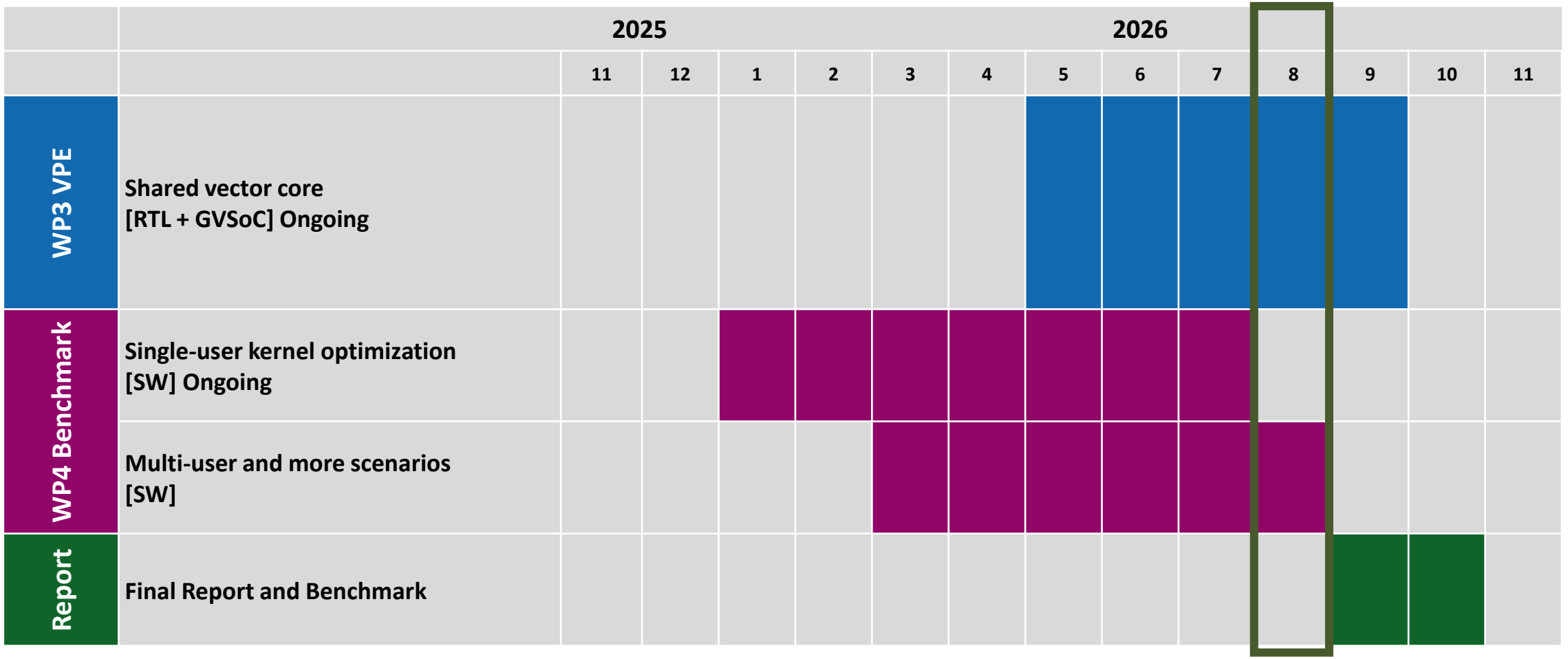
Next Step

- **Diyou Side**
  - Physical design exploration for multi-group cluster => help to determine the BW we can have
    - First PD run looks very positive => we can increase BW between groups
  - Rebase onto main (folded cache work) and merge
  - Multi-Snitch cores sharing one Spatz
    - Plan
      Add additional Snitch into the current CC (what we have done in a different project)
    - Some needed additions
      1) Snitch inside the tile should all connect to the Spatz, using muxing to select the host
      2) Software and runtime support to reduce the coding complexity (some helper functions)
- **Zexin Side**
  - GVSoC development and calibration under multi-group config
  - SW development based on the benchmark plan

Timeline

		2025									2026			
		11	12	1	2	3	4	5	6	7	8	9	10	11
WP1 ManyCore Architecture	Multi-Tile Scaling [RTL] Complete													
	4/16-Group Scaling [GVSoC + RTL] Ongoing													
	Interco. Optimization (multi-group) [RTL] Ongoing													
	Timing optimization for NoC [RTL] Ongoing by colleague													
	NoC Topology Exploration [RTL, GVSoC] Ongoing by student													
WP2 Cache	Cache partitioning [RTL] Complete													
	WT-L1 by master student [RTL] Complete, Extending Ongoing													
	Folded Cache [GVSoC + RTL] Complete, Opt Ongoing													

Timeline


2025
2026
11
12
1
2
3
4
5
6
7
8
9
10
11
WP3 VPE
Shared vector core
[RTL + GVSoC] Ongoing
WP4 Benchmark
Single-user kernel optimization
[SW] Ongoing
Multi-user and more scenarios
[SW]
Report
Final Report and Benchmark

L1 Access Latency

- **Problem: increased L1 access latency when scaling up**
- **Solutions**
  - **(Ongoing, RTL & Physical) Hardware optimization**
    Further optimize the interconnection and cache controller to reduce access latency on existing architecture
    => Retiming the current interconnection, remove unnecessary cuts in interco and cache
    => Target to reduce 1-2 cycles (~5 cyc) for hit-to-use latency
    Currently has reduced to 6 cycles:
    Core => Xbar => AMO => Ctrl (2) => AMO => Core
  - **(Complete, RTL) Cache partitioning**
    Partition the shared L1 to keep data localized to avoid remote access latency
  - **(Complete with some problems, RTL & Physical) WT-L1**
    Explore and evaluate a private WT L1 for each core
    => Performance degrade on single tile
    => Needs more work to support larger system

L1 Access Latency

- **Problem: large NoC latency (~28 cyc based on previous exploration)**
- **Solutions**
  - **(Ongoing, Physical) Reduce hop latency**
    Explore the timing under 7nm technology to see the possibility of reducing per-hop latency to 1 cycle
    => Reduce the latency by 1/2
  - **(Planned, Physical) Cut clk-domain (optional)**
    If 1-cyc is not possible, explore to operate the NoC under a higher frequency than cores (e.g. 1.5GHz)
    => Effectively cut the latency by 1/3
  - **(Planned, GVSoC & Physical) NoC Topology**
    Explore different NoC topologies
    => Torus to reduce the diameter of NoC by 1/2, reduce longest latency by 1/2

PE-Related

- **Problem: Improve scalar core performance**
- **Solutions**
  - **(Planned, RTL & GVSoC) Shared vector core**
    Explore two Snitch sharing one Spatz configuration to improve scalar performance
    Software complexity needs to be considered, especially on the possible conflicting use of register file.
    Discussed Solution: critical section

RLC Workload

- **Problem: Current RLC model cannot reveal all properties of the kernel**
- **Solutions**
  - **(Ongoing, SW) Kernel optimization**
    Adapt the current model to better represent the real cases, e.g. do some operations on the data. Further cooperation with HQ side to develop a more complete and better model.
  - **(Planned, SW) Kernel development**
    Implement the missing use cases for multi-user

Thank you!